Algorithmic Robotics
COMP/ELEC/MECH 450/550
Project 4: Out of Control Planning

The documentation for OMPL can be found at http://ompl.kavrakilab.org.

In the previous project, you computed collision-free geometric paths for the Panda arm without modeling velocity, acceleration, or torque limits. In this project, you will be planning for more complex systems – systems with dynamical constraints on their motion. For these complex systems, a collision-free path may not always be feasible. As an example, consider a car trying to parallel park. Although a *straight-line* path sideways to the parking spot may be collision free, the car cannot translate sideways and must execute a longer path to reach the parking spot. The fact that the car cannot directly move sideways implies that the car is a *non-holonomic* system; the system cannot control its position directly, and has constraints that include terms other than its position. This is a key point that differentiates planning between geometric and dynamic problems.

In this project you will plan motions for non-holonomic systems whose dynamics are described by an ordinary differential equation of the form $\dot{q} = f(q,u)$, where q is a vector describing the current state of the system and u is a vector of control inputs to the system. The systems you will plan for are a torque-controlled pendulum and a car-like vehicle moving in a street environment. Dynamically feasible and collision-free motions for these systems will be computed using the RRT and KPIECE planners that are already implemented in OMPL. Additionally, you will also learn about and implement a new planner called Reachability-Guided RRT (RG-RRT), one of many variants of RRT.

Use your Project 2 and Project 3 OMPL environment and the starter in `template/`:

```
micromamba activate ompl_env
cd project4_2026/template
make
```

Read the README for build, run, visualization, and benchmark commands.

A Torque-controlled Pendulum System

A pendulum is one of the simplest dynamic systems. The state $q = (\theta, \omega)$ of the system is described by the orientation of the pendulum (θ) and its rotational velocity (ω). Assume that $\theta = 0$ corresponds to the bar of the pendulum being horizontal. The objective is to move the pendulum from an initial state of hanging down at rest to a final state of pointing up at rest:

$$(\theta_s, \omega_s) = (-\tfrac{\pi}{2}, 0) \qquad \rightarrow \qquad (\theta_g, \omega_g) = (+\tfrac{\pi}{2}, 0)$$

A motor can apply a torque τ to the axis of rotation of the pendulum. The pendulum's dynamics are described by the following differential equation (g is gravity, ≈ 9.81):

$$\dot{q} = \begin{pmatrix} \dot{\theta} \\ \dot{\omega} \end{pmatrix} = \begin{pmatrix} \omega \\ -g\cos\theta + \tau \end{pmatrix}$$

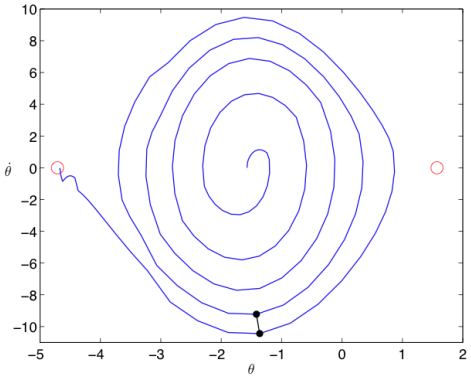


Figure 1: A possible solution trajectory for the pendulum in phase space. *From Shkolnik et al.; see below.*

When an arbitrarily large torque can be applied, the planning problem is not difficult as the torque can hold the system *quasi-statically* at any configuration. Many real manipulators use this strategy and utilize very large torques, allowing them to plan geometrically. However, is this always the case? In reality, the pendulum's motor can source only a finite torque.

In your implementation, use 3, 5, and 10 as the absolute value of the upper and lower bounds on the torque limits (i.e., $\tau \in [-3, 3]$ for a torque bound of 3). Use 10 as absolute value of the rotational velocity limit. Figure 1 shows a possible solution on the (θ, ω) phase space of the pendulum. The start pose is at the center of the spiral and either one of the red circles are valid end poses.

A Car-like system

The second system involves a simple vehicle moving in a street environment, an example of which is shown in Figure 2. The state of the vehicle is represented by its position (x, y), heading θ, and forward velocity v:

$$q = (x, y, \theta, v)$$

Note that forward velocity *can* be negative, in which case the car moves in reverse. The control inputs to this system u consist of the angular velocity ω and the forward acceleration of the vehicle $\dot{v}$,

$$u = (\omega, \dot{v})$$

Both terms of u are bounded in magnitude. The system dynamics are described by:

$$\dot{q} = \begin{pmatrix} \dot{x} \\ \dot{y} \\ \dot{\omega} \\ \dot{v} \end{pmatrix} = \begin{pmatrix} v\cos\theta \\ v\sin\theta \\ \omega \\ \dot{v} \end{pmatrix}$$

Similar to the pendulum system, the car cannot source an arbitrarily large acceleration or angular velocity. You need to set bounds on the control space to ensure a dynamically feasible trajectory.

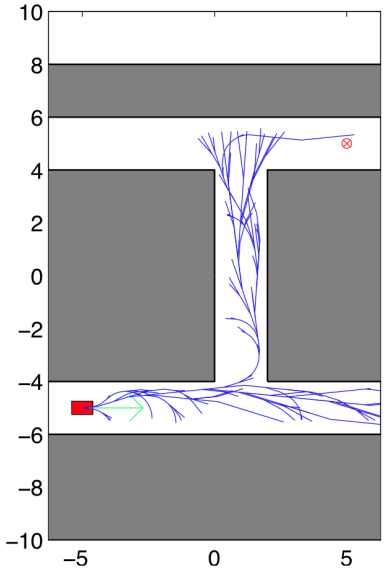


Figure 2: Vehicle navigating in street environment. *From Shkolnik et al.; see below.*

Planning for Dynamical Systems

The key difference when planning for dynamical systems is that the *input space* of controls must be sampled (a *control space*), along with a duration to apply the control. In geometric planning, local motions interpolate configurations without modeling dynamics. For dynamical systems, a control input is applied for a prescribed time, and the equations of motion are applied to compute the resulting configuration. The planners in OMPL for dynamic systems sample controls that are applied for some small period of time. It is thus necessary to integrate the equation $\dot{q} = f(q,u)$ to get the resulting state. OMPL includes support for numerical integration and you "only" have to implement $f(q,u)$. A detailed example is given at http://ompl.kavrakilab.org/odeint.html.

Construct each system's state space and a correct `StateValidityChecker`. For the pendulum system, you can assume an environment without obstacles. However, the state validity checker must ensure that the angular velocity of the pendulum is within the bounds that you specify. Similarly you must verify that the velocity of the vehicle is within the bounds you set. For simplicity, you can assume that the vehicle is a point system for collision checking purposes, but the 2D collision checking methods from Project 2 can also be used to add geometry to the vehicle if you wish.

See the OMPL demo programs for some examples of how to put all the different pieces together (available online on OMPL website, on the "demos" page). Look in particular at `Rigid body planning with ODESolver and controls`.

Reachability-guided RRT

An article by Shkolnik et al. (2009) proposes a motion planning algorithm for dynamic systems. The authors propose that an RRT should only be grown towards a random state that is likely to be *reachable* from its nearest neighbor in the tree. For dynamic systems, states that are near a given state are not always easily reachable. For instance, vehicle systems cannot easily move sideways. You will use the information in their article (summarized below) to understand the problem and the performance characteristics of the systems they assessed. Next, you will implement the motion planner they describe, the *reachability-guided* RRT (RG-RRT) and test its performance.

The main change of RG-RRT over RRT is that for each state q in the tree, an approximation of the *reachable set* of states, $R(q)$, is maintained. The reachable set $R(q)$ is the set of states the system can arrive at, starting at state q, after applying any valid control(s) for a small period of time (a fixed value you set). You can approximate $R(q)$ by choosing a small number of controls and computing the states that were reached after applying them for a short duration. For the pendulum you should pick 11 evenly spaced values for τ between the torque limits (i.e., {-10, -8, ..., 8, 10}). For the car you can do the same for u_0 (you can ignore u_1 for the reachable set). Next, apply each of the controls to the start state q for a small time step to obtain $R(q)$ (use the `SpaceInformation::propagate` method for this). You can store the reachable set approximation by extending `Motion` objects (such as those in RRT) to also store the reachable set (as a vector of states). You can assume for this project that the control space is a `RealVectorControlSpace` with bounds. Please make sure you write your assumptions in your report.

The next step is to determine which node to choose to expand the tree. Given a random state q_{rand}, you need to first find the state q_{near} that is closest to q_{rand}. Then, you must check if there exists a state in $R(q_{\text{near}})$ that is closer to q_{rand} than q_{near}. If q_{near} is closer to q_{rand} than any state in $R(q_{\text{near}})$, discard q_{rand} and repeat. Otherwise, q_{near} should be chosen to expand the tree.

To implement RG-RRT, start with OMPL's control-based RRT implementation: https://github.com/ompl/ompl/tree/main/src/ompl/control/planners/rrt.

Projections

KPIECE (among other planners) requires the definition of a projection for the state space being planned in. KPIECE uses the projection to estimate how well different parts of the state space have been sampled. For the existing state spaces in OMPL a projection is already defined; however, a good projection often requires some information about the specific robot and task. Although projections are generally a reduction in dimension, projections in OMPL do not have a limit on how many dimensions you wish to define nor do they require that you chose a subset of the dimensions in your state space.

For example, one of the dimensions in your projection could even be the Cartesian position of some point on the robot. Often times, a desirable projection might be one which has a strong correlation with progress toward the goal or indication of greater coverage in the workspace. With control spaces, such a projection usually includes some information from the dynamic dimensions in the state.

Project exercises

1. Implement the state validity checker and differential equations for the pendulum and the vehicle systems described above. Solve the motion planning problems described for these systems using the RRT planner. Visualize the solution paths to make sure they are correct and to compare the solution paths found using torque values of 3, 5, and 10 for the pendulum problem.
2. Extend the program from #1 to solve the pendulum and the vehicle problems using the KPIECE planner. You will need to define a projection for the state spaces you create for the pendulum and the car. See http://ompl.kavrakilab.org/projections.html for details on how to define a projection and associate it with a state space.
3. Implement RG-RRT (see Shkolnik et al., 2009) and solve the pendulum and vehicle problems as in #1 and #2. Make sure to visualize the solution paths.
4. Compare your RG-RRT against RRT and KPIECE using the OMPL `Benchmark` class for both the car and pendulum environments. Use a torque value of 3 for the pendulum. Any conclusions must come from at least 20 independent runs of each planner.

Protips

- Be sure to use the `ompl::control` namespace instead of `ompl::geometric` in this project. This includes the `SimpleSetup` class and all of the planners.
- Solution paths from the *control-based* planners in OMPL contain more information than their geometric counterpart. You can "geometrize" a controlled path using the `asGeometric` method. This returns a `PathGeometric` object; export its states with `printAsMatrix()` to plot the pendulum's phase-space path or the car's path over the street obstacles.
- Make sure to perform state validity checking for the reachable set in your RG-RRT implementation. If a state is not valid, it certainly is not *reachable*.
- It is *highly* unlikely that a control-based planner will be able to find a goal configuration exactly. Think about why this is the case. The call to `SimpleSetup.setStartAndGoalStates()` has a third parameter that defines an acceptable radius around the goal state. Any state that is within this radius will be considered an exact goal state. You will need to play with this parameter to successfully plan.

- In this exercise you will have to generate new states for the reachability set of a state. To do this, you want to apply some control to an existing state. You can allocate controls by allocating new controls from `ompl::control::SpaceInformation` using `allocControl`. Just like with states, you can cast controls to their control type defined in the control space. For example, `RealVectorControlSpace` has `RealVectorControlSpace::ControlType`. Use `ompl::control::SpaceInformation` to generate new states through `propagate`, which applies a control to get a new state.
- The starter's 2D collision checkers accept numeric coordinates. Extract x and y from the car's state and pass them to `isValidPoint`; for a square vehicle, also pass its heading and side length to `isValidSquare`. Check velocity and state-space bounds separately.

Deliverables

This project can be completed in groups of up to 3 people. **We strongly encourage groups of 2 or 3 people - this is a hard assignment.** With your submission, please include the contribution of each member of the group. When grading this project, we may ask groups to come and talk to the TAs and the instructor. In that case we will ask each member of the group questions related to their individual contributions.

1. We are setting a check point **due Thursday October 29, 2026 at 1pm**. You need to finish Project exercise 1 and 2. That means you need to fill in `Project4Car.cpp` and `Project4Pendulum.cpp`, except for the part related to RG-RRT and benchmarking. Your code should be able to do motion planning with RRT and KPIECE with the pendulum and the vehicle systems. You can also submit what you have implemented for RG-RRT, even it is half-done, but make sure your code compiles. You are also expected to attach a report in PDF format, no more than 1 page, showing the images of your environments with example paths generated by RRT and KPIECE. Make sure you describe the images clearly.

2. **The final submissions are due Tuesday November 10, 2026 at 1pm**. At this point, you are expected to finish the whole project, and submit the final report. Your code can change from what you submitted for the check point, as for the check point, we grade more on your effort, not the quality. If you are a graduate student, you have finished this project.

3. For **undergraduate students** only, you will need to submit the slides and recording of your presentation **due Tuesday November 17, 2026 at 1pm**. Please submit the slides on Canvas, and email your video recording of the presentation to the TAs. The presentation should be at most 10 slides and 5-7 mins long. It should concentrate on the solution and description of the results. Feel free to include what you think you have done well and/or what you found difficult in this project. Note the graduate students are not required to do presentation/slides for this project. If your team has both undergraduate and graduate students, the graduate students are welcome to help with the presentation/slides.

To make the **check point** submission, clean your build space with `make clean`, zip up the project directory into a file named `Project4_Checkpoint_<your NetID>_<partner's NetID>.zip`, and submit to Canvas. To make the **final** submission, clean your build space with `make clean`, zip up the project directory into a file named `Project4_<your NetID>_<partner's NetID>.zip`, and submit to Canvas. Your code must compile and run in a modern Linux environment with the course's OMPL environment. If you deem it necessary, also include a README with details on compiling and executing your code. In addition to the archive, submit a short final report that summarizes your experience from this project. The report should be anywhere from 1 to 5 pages in length in PDF format, and contain the following information:

- **(4 points)** A succinct statement of the problem that you solved.
- **(2 points)** Images of your environments and a description of the start-goal queries you are evaluating.
- **(2 points)** A short description of the robots (their geometry) and configuration and state spaces.
- **(10 points)** Explain the differences in solution paths when torque is limited to 3, 5, and 10 for the pendulum problem.
- **(10 points)** A summary of benchmarking data for the RRT, KPIECE and RG-RRT planners for both systems. Use a torque value of 3 for the pendulum for the other benchmark and comparison exercises. This data must be presented in a quantitative manner (i.e., plots or tables) and any conclusions must be supported from this data. Consider the following metrics: computation time, path length, number of tree nodes, and success rate. When referencing benchmarking results you must include the referenced data as figures.
- **(10 points)** A head-to-head comparison of RRT and RG-RRT. Discuss the performance trade-offs of RG-RRT for computing reachable states in terms of the *length* of the time period and the *number* of controls. Support your claims with images and/or benchmark data.
- **(2 points)** Rate the difficulty of each exercise on a scale of 1–10 (1 being trivial, 10 being impossible). Give an estimate of how many hours you spent on each exercise, and detail what was the hardest part of the assignment.

Additionally, you will be graded upon your implementation. Your code must compile, run, and solve the problem correctly. Correctness of the implementation is paramount, but succinct, well-organized, well-written, and well-documented code is also taken into consideration. Visualization is an important component of providing evidence that your code is functioning properly. The breakdown of the grading of the implementation is as follows:

- **(35 points)** RG-RRT
- **(10 points)** Pendulum Problem
- **(10 points)** Car Problem
- **(5 points)** Benchmark

Each team need only provide one submission.

Reference

A. Shkolnik, M. Walter, and R. Tedrake, Reachability-guided sampling for planning under differential constraints, in *IEEE Intl. Conf. on Robotics and Automation*, pp. 2859–2865, 2009. http://dx.doi.org/10.1109/ROBOT.2009.5152874, http://dspace.mit.edu/openaccess-disseminate/1721.1/61653